SCSR2043
OPERATING SYSTEMS

Introduction 1 & 2 - Concept-first study guide

A connected explanation of the first two lecture introductions, based on the Week 1-2 materials. Covers the kernel, CPU, buses, registers, buffers, controllers, interrupts, RAM, processes, protection, OS structures, and computing environments.

How to read
Start with the big picture and the hardware section. Pause at each diagram and say out loud what each part does. The final worked example and practice questions test whether you can connect the concepts, rather than just recite definitions.

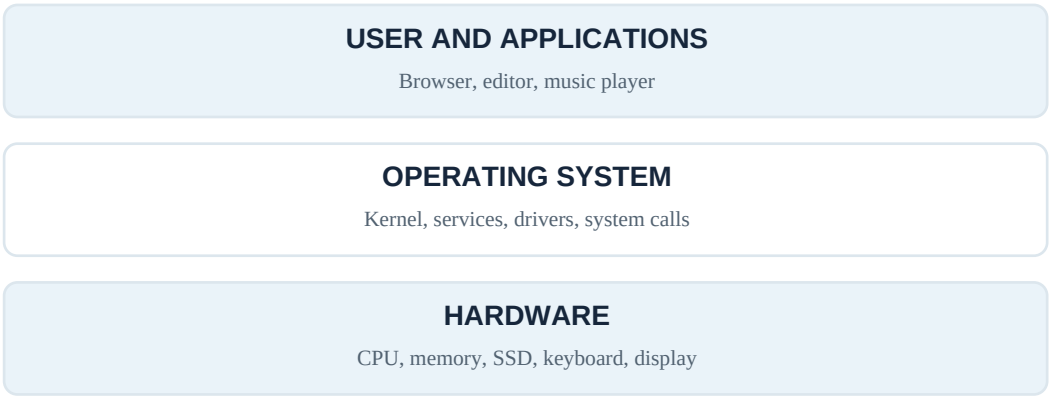

Figure 1. Apps ask for services; the OS coordinates hardware access.

Remember one sentence: An operating system is software that manages computer resources and provides a safe way for programs to use them.

Roadmap: what the chapter is teaching

These ideas form one story: the CPU executes instructions; programs share memory and devices; the operating system controls that sharing; hardware signals and privilege rules help the OS stay in control.

Area	Questions to ask
OS and kernel	What work must be centralized? Why can't every app control hardware directly?
CPU and connections	What are registers, buses, controllers, drivers, and buffers?
Interrupts and I/O	How can devices finish work while the CPU does something else?
Storage and memory	Where is data now? How is a cache different from a buffer?
Execution and protection	How can many programs run? How does a system call reach the kernel?
Systems and environments	How do cores, clusters, real-time systems, VMs and clouds differ?

Learning method

- Read each section slowly, and cover the definitions before attempting the explanation in your own words.
- Draw the CPU-RAM-controller diagram from memory. Label the hardware and the software.
- Try the save-file walkthrough without looking. If it makes sense, most core terms will make sense too.

A warning about analogies
They help you start, but they are not precise engineering definitions. For example, saying the kernel is a manager does not mean it is one physical chip or that it personally performs every instruction.

1. What an operating system does

The OS coordinates limited hardware resources. Many applications might need the CPU, RAM, network, storage, display, or speakers at the same time. The OS decides how they get these resources, separates one process from another, and provides common functions such as reading files.

User view versus system view

User view	System view
Open apps, save files, use the keyboard, hear audio, get a responsive computer.	Allocate CPU time and memory; handle devices; enforce permissions; recover from errors; keep programs isolated.

What exactly is the kernel?

The **kernel** is the central, privileged part of the OS. It typically handles scheduling, memory management, interrupts, device control, and protection. It is software: CPU cores execute kernel instructions when kernel work is needed.

The **whole operating system** usually includes much more than the kernel: system utilities, libraries, background services, drivers, and often user interfaces. Exact boundaries vary by design. A desktop app is not itself part of the kernel.

Think about it
Your editor requests a file save. The editor knows the text, but the OS checks access and coordinates storage. The CPU executes whichever instructions are running, whether from the app or from the kernel.

2. The CPU, registers, and the bus

CPU: the instruction executor

The CPU runs machine instructions. Each core typically fetches an instruction, decodes what it means, performs the operation, and moves to the next one. A CPU with multiple cores can execute more than one instruction stream at the same time.

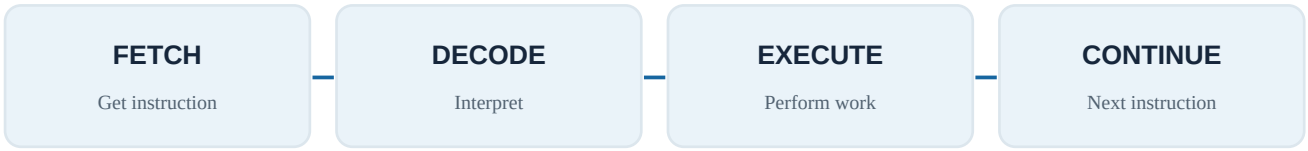

Figure 2. A simplified instruction cycle.

Registers: immediate working storage

Registers are tiny, fast storage locations inside a CPU core. General-purpose registers can hold operands, addresses, and intermediate results. Specialized registers include the **program counter** (the instruction address to execute next) and **stack pointer** (tracks the current stack).

For example, addition might load 7 and 5 into registers, add them, and hold 12 in a register. A later instruction might store that result in memory. This does not mean every calculation automatically writes to RAM immediately.

Bus or interconnect: a communication path

A bus is a pathway for data and control information between computer components. Introductory descriptions distinguish **data** (what value), **address** (which location), and **control** (read/write and other signals). Modern hardware has multiple interconnect designs, so do not imagine one literal wire used for everything.

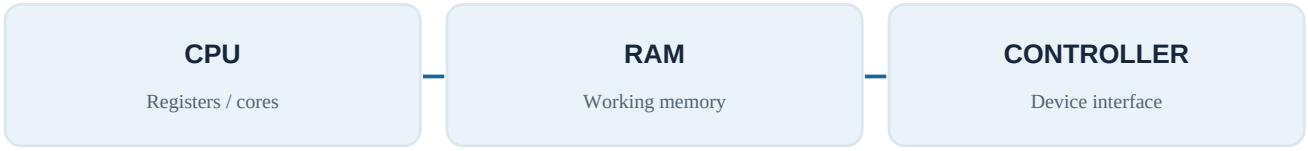

Figure 3. Simplified communication across interconnects.

3. Controllers, drivers, and buffers

Controller is hardware; driver is software

A **device controller** is hardware or device electronics that accepts commands and reports status. A **device driver** is software that knows how to operate that controller. An I/O **device** is the hardware that sends or receives data, such as an SSD, keyboard, or network adapter.

Term	Location	Role
Controller	Hardware	Accept commands; expose status; coordinate device activity.
Driver	Software	Send controller commands; manage completion and errors.
Device	Hardware or virtual	Produce or consume input/output data.
Bus / interconnect	Communication hardware	Move data, addresses, or control signals.

What is a buffer?

A **buffer** is temporary storage used while data is moving, accumulating, or waiting to be processed. Buffers are often in RAM or on a controller. They are useful when a producer and consumer run at different speeds.

Streaming video example: the network delivers data in bursts, but playback needs data continuously. A playback buffer holds some bytes in advance. During a short interruption, playback can continue using buffered data. If the interruption lasts too long, the buffer empties and playback stalls.

Do not mix these up
A register is immediate CPU working storage. A buffer holds data in transit. A cache holds copies to avoid repeating slower accesses. RAM is general working memory. These words describe different roles; a buffer or cache may physically live in RAM.

4. Interrupts and I/O operations

Why interrupts exist

A program may request data from storage. If the CPU checked continuously whether the drive had finished, much CPU time could be wasted. An **interrupt** is a signal/event that can cause the CPU to pause its current execution, save enough state, and run an appropriate handler.

Typical sequence: an I/O request begins; the CPU can run other ready work; the device reports completion; the CPU enters an interrupt handler; the OS records the result and makes waiting work runnable. In some designs, the system may poll instead of, or alongside, using interrupts.

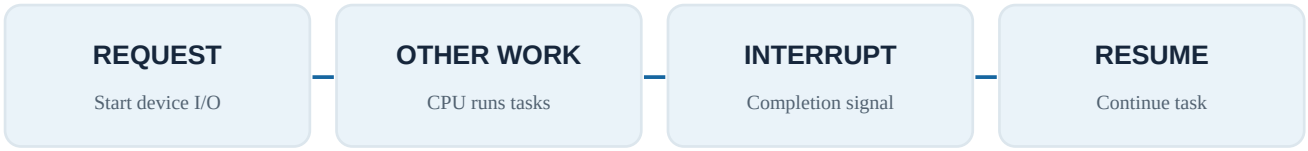

Figure 4. One possible device I/O and completion flow.

Trap, exception, and system call

An **exception** is associated with execution of an instruction, such as division by zero or an invalid memory access. A **system call** is a controlled way for a user program to request an OS service. A **hardware interrupt** commonly comes from a device or timer. Exact terminology and hardware mechanisms vary.

Direct Memory Access (DMA)

DMA allows suitable controllers to move blocks of data to or from memory without having the CPU copy every byte in a tight loop. The CPU still sets up the transfer and the OS still manages memory, permissions, and completion. DMA is especially useful for larger I/O transfers.

Method	Basic idea
Busy waiting / polling	Repeatedly check whether an operation has finished. Appropriate in some low-latency designs; expensive if waiting for a long time.
Interrupt-driven I/O	Let hardware signal when attention is needed. The CPU can do other work until then.
DMA	Move blocks between memory and a capable device/controller with less CPU copying.

5. RAM, storage, caches, and persistence

The closer data is to the CPU's execution units, the faster it is often accessible, but the smaller and more expensive the storage tends to be. These are general tendencies, not universal rules.

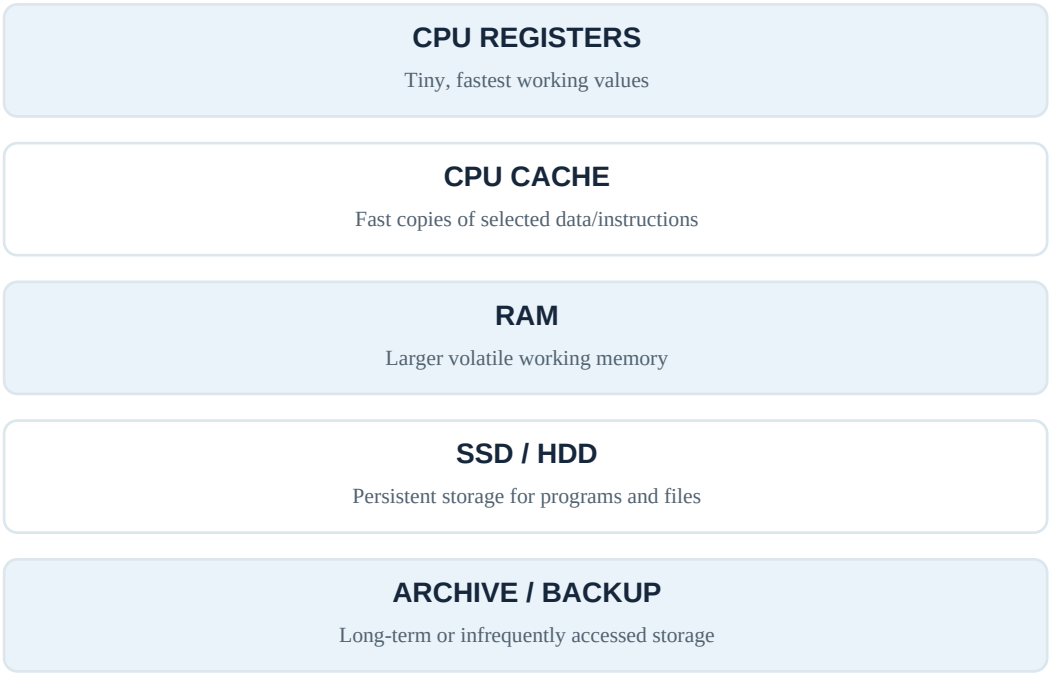

Figure 5. Conceptual memory and storage hierarchy.

Volatile vs nonvolatile

Volatile memory generally loses its contents without power (ordinary RAM, registers, and caches). **Nonvolatile storage** generally keeps its contents without power (SSD, HDD, flash). Programs normally load code and active data from persistent storage into RAM for execution.

Cache hit, miss, and coherence

A **cache hit** means requested data is found in the faster cache; a **miss** means the system must fetch it elsewhere. Multiple cores may hold copies of the same memory line. Hardware cache-coherence mechanisms help those cores agree about writes and visibility under the machine's memory model.

The term **consistency** concerns which memory-operation orders software may observe. It is related to, but not identical with, cache coherence. A copy being coherent does not automatically mean every operation occurs in the simple order you imagined.

Check yourself
When opening a saved photo, the SSD supplies persistent bytes, RAM holds active program data, CPU cache may hold recently accessed pieces, and registers hold immediate values for instructions.

6. Computer-system organization

Single processor, multicore and multiprocessor

A system may have one processor with one core, one processor with multiple cores, or several processor packages, each perhaps with multiple cores. Parallel processing means multiple execution units can perform work at the same instant.

Symmetric multiprocessing (SMP) generally means multiple processors/cores have a broadly shared memory and OS arrangement. An OS must schedule work and coordinate concurrent access to shared state.

Clustered systems

A **cluster** comprises separate computers (nodes) connected by a network and working together. Unlike cores in one CPU, cluster nodes are separate machines with their own memory and operating systems. Clustering can support performance, availability, or both, depending on design.

Concept	What it describes
Multicore	Multiple processing cores within a processor/package.
Multiprocessor	Multiple processing units/packages sharing work in one system.
Cluster	Multiple networked computer nodes collaborating.
Parallelism	Two or more tasks literally executing simultaneously.
Concurrency	Multiple tasks making progress over overlapping time intervals.

7. Programs, processes, and threads

Program versus process

A **program** is stored instructions and data, such as an executable on disk. A **process** is an executing instance with state and managed resources: an address space, open files, security identity, and one or more threads. Two processes may run the same program without becoming the same process.

Thread: an execution path

A **thread** is one flow of execution within a process. Threads typically share their process's memory and resources, while retaining their own instruction state and stack. Sharing makes cooperation easier but creates synchronization problems when two threads update the same state.

Why context switching happens

The OS scheduler selects runnable threads or processes. A **context switch** saves execution information for one task and restores information for another. A task can stop because it waits for I/O, yields, is preempted by a timer, or because a higher-priority task needs to run.

Figure 6. An example cycle through task states.

On a single core, the browser and music player can appear to run together because switching is fast. That is **concurrency**. Two separate cores can actually execute instructions at once, giving **parallelism**.

8. User mode, kernel mode, and protection

General-purpose systems usually separate **user mode** from a more privileged **kernel mode**. Applications run with limited privileges. They cannot directly perform arbitrary sensitive operations such as configuring hardware controllers or mapping any physical memory they choose.

Figure 7. Protected transition between application and kernel work.

System calls

An app might request open(), read(), write(), or other OS services. The request passes through an OS interface, CPU privilege changes under controlled rules, kernel code checks parameters/permissions and does the work, then returns a result. A system call is not the same thing as a normal user-space function call.

Why a timer matters

If an application loops forever, the OS still needs a way to regain control. A hardware timer can periodically interrupt execution, allowing the OS to make scheduling decisions. This helps enable preemptive multitasking.

Protection and security

Protection concerns rules on who may access which resources. **Security** includes protecting the system against unwanted use and attacks. The OS can enforce process isolation, access rights, authentication, and auditing, though the exact mechanisms depend on the system.

Important distinction
An interrupt does not mean the CPU is physically switched off. It changes the flow of executed instructions so an event can be handled; afterwards the previous task may resume.

9. How OS services are organized

System programs and services

Operating systems provide file tools, loaders, shells, graphical environments, network services, and other utilities. Some run in user space; others may interact closely with the kernel. You should not treat every background service as a part of the privileged kernel.

Kernel design approaches

Approach	Idea / tradeoff
Monolithic kernel	Many core OS facilities run in kernel address space. Calls between components can be direct, but bugs have broad impact.
Microkernel	Keeps a smaller privileged core; moves more services into user processes. Often needs more inter-process messaging.
Layered design	Organizes software into abstraction layers with defined responsibilities.
Modular / hybrid	Mixes principles; may load modules or use varying placements of services.

Real operating systems rarely fit a classroom category perfectly. The useful question is which component runs where, what privilege it has, and how it communicates.

OS responsibilities at a glance

- **Process management:** create, schedule, synchronize and terminate tasks.
- **Memory management:** allocate and isolate address spaces, track free/used memory, manage virtual memory.
- **File/storage management:** names, directories, metadata, allocation and persistence.
- **I/O management:** drivers, buffering, caching, device access and recovery.
- **Protection:** permissions, identities and resource boundaries.

10. OS types and computing environments

Type	Defining concern / example
Batch	Processes jobs with little interactive involvement.
Time-sharing / interactive	Keeps multiple users or applications responsive through scheduling.
Real-time	Correctness can depend on meeting deadlines, not just average speed.
Embedded	Runs inside purpose-built hardware; may have tight resource constraints.
Mobile	Combines general OS features with battery, sensors and app isolation.
Distributed / networked	Coordinates computation and resources across connected systems.

Real-time does not mean 'very fast'

A real-time system is designed around timing guarantees. A result delivered after its deadline can be a failure even if the computation was fast on average. The strength of guarantee depends on whether the system has hard or soft real-time requirements.

Virtual machines and hypervisors

A **hypervisor** enables multiple virtual machines to share host hardware while presenting virtualized hardware to their guest OSs. A VM typically includes its own guest OS. A **container**, by contrast, commonly shares the host kernel while isolating applications using OS features.

Cloud service models

Model	What you usually manage
IaaS	Guest OS, installed software and applications, while provider operates underlying infrastructure.
PaaS	Application and configuration; provider manages much of the runtime/platform.
SaaS	You use a completed software service; the provider operates the application.

Exam trap
Cloud is a way computing resources are delivered; virtualization is one technology that can help deliver them. They are related, but not synonyms.

11. Worked example: saving a file

This walkthrough joins the terms into one flow. Suppose you press Save in a text editor. The implementation varies by OS and file system, but the roles remain useful.

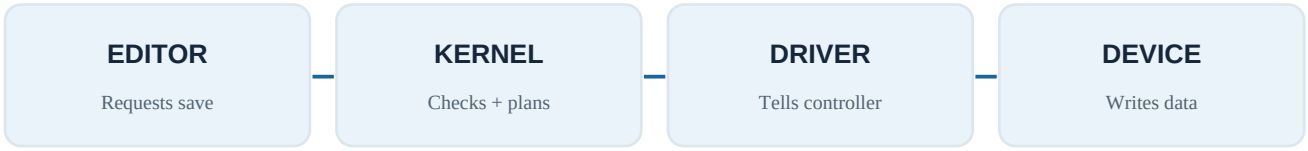

Figure 8. Simplified software-to-hardware save path.

- **1. Application:** the editor has text in its process memory and requests a file write through an OS API or system call.
- **2. Transition:** execution enters a controlled kernel path. The OS checks the file descriptor, access rights, and request arguments.
- **3. Memory:** bytes may be copied or mapped to OS-managed buffers and cached file-system pages in RAM.
- **4. File system:** the OS maps the file and offset to storage metadata and decides what blocks must change.
- **5. I/O:** a storage driver sends requests to a device controller; data may travel by DMA between RAM and the device.
- **6. Scheduler:** while the write is pending, the task may sleep so another ready task can use the CPU.
- **7. Completion:** the device may signal an interrupt. The OS completes bookkeeping and wakes waiting work.
- **8. Return:** the application receives the result. A write call succeeding does not always guarantee durable power-loss-safe storage: synchronization or flushing may be needed.

Term	Where it appeared
Kernel	Checked access, arranged OS work and handled completion.
Register	Held immediate CPU execution state and operands.
Bus / interconnect	Carried messages/data among CPU, RAM and controllers.
Buffer	Temporarily held bytes waiting to move or be consumed.
Interrupt	Reported hardware completion.
Scheduler	Chose work while the editor waited.

12. Common confusions and quick comparisons

Confused terms	A reliable distinction
Kernel / CPU	Kernel = privileged software; CPU = hardware executing instructions.
Bus / buffer	Bus = communication pathway; buffer = temporary data storage.
Register / RAM	Register = tiny CPU-local working storage; RAM = main working memory.
Cache / buffer	Cache = copy for faster future reuse; buffer = holding data while moving/waiting.
Driver / controller	Driver = software; controller = device electronics.
Process / program	Process = running instance with state; program = stored code.
Interrupt / system call	Interrupt = external/timer event; system call = program requests privileged service.
Concurrent / parallel	Concurrent = overlapping progress; parallel = literally simultaneous execution.
Multicore / cluster	Multicore = cores in computer; cluster = multiple networked computers.
VM / container	VM commonly has a guest OS; container commonly shares host kernel.

One-minute self-explanation

A program runs as a process. The scheduler chooses a runnable thread. The CPU executes instructions using registers and accesses RAM through its memory system. If it needs a file or network data, the program asks the OS. The kernel checks access and coordinates a driver and device controller. Data may move via interconnects and buffers. A later interrupt can signal completion and allow the waiting process to continue.

13. Test yourself: explain, do not just recall

Short-answer questions

1. Explain the kernel versus the entire operating system, and give one kernel function.

2. If a CPU adds two numbers, where might the immediate intermediate value be stored?

3. Separate bus, controller, driver and buffer with one sentence for each.

4. Why do interrupts help CPU utilization when storage or network devices are slow?

5. Give one example each of register, buffer and cache use.

6. Can one core run several programs without literally executing two instructions simultaneously?

7. Distinguish program, process and thread.

8. Explain what happens when a user-space app calls a privileged OS service.

9. Why do user/kernel modes and hardware timers help protect multitasking?

10. What is a real-time deadline, and why isn't 'fast on average' enough?

11. Differentiate multicore, cluster and virtual machine.

12. During Save, which components handle scheduling, memory, file structure and device transfer?

Multiple-choice questions

- 13. Which generally remains stored without power? A Register B CPU cache C SSD D RAM
- 14. Device I/O completion may be reported with: A context switch B hardware interrupt C program counter D RAM refresh
- 15. Several tasks make progress through rapid switching on one core: A parallelism B concurrency C clustering D DMA
- 16. Which is software? A SSD controller B RAM chip C CPU core D device driver
- 17. In which cloud model does the customer commonly administer the guest OS? A SaaS B PaaS C IaaS D none
- 18. Which is correct? A cache always holds permanent data B buffering guarantees faster internet C system calls can enter kernel code D real-time means maximum CPU speed

14. Explained answer key

1. The kernel is the privileged core; the OS is broader and includes services and utilities. Scheduling is a kernel responsibility.

2. Usually in a CPU register first. Cache and RAM updates follow the relevant instructions and memory rules.

3. Bus: pathway. Controller: device hardware interface. Driver: software that operates it. Buffer: temporary holding area.

4. The CPU can run another ready task while I/O is ongoing; a completion interrupt can signal when attention is needed.

5. Register: arithmetic operand. Buffer: queued stream bytes. Cache: recently used copied data.

6. Yes. Rapid scheduling produces concurrency on a single core without simultaneous independent instruction execution.

7. Program = stored instructions; process = active instance/resources; thread = execution path within a process.

8. A controlled system-call entry changes privilege, kernel validates and carries out work, then returns a result to user mode.

9. Privilege modes stop arbitrary app access. Timers let the OS regain control when a program would otherwise keep running.

10. A timing deadline is a required completion time. Late output may be useless regardless of good average speed.

11. Multicore = multiple cores, one computer. Cluster = several networked nodes. VM = virtualized machine with guest environment.

12. Scheduler manages runnable work; memory management manages buffers; file system organizes files; driver/controller perform I/O.

13. C, SSD: nonvolatile storage.

14. B, hardware interrupt: one common I/O completion signal.

15. B, concurrency: work overlaps over time; parallelism requires simultaneous work.

16. D, device driver: executable software.

17. C, IaaS: customers commonly manage the guest operating system.

18. C, a system call can invoke privileged OS services.

Your checkpoint

Ready for the next lecture?
Explain the Save workflow in your own words and correctly distinguish kernel/CPU, driver/controller, register/buffer/cache, and concurrency/parallelism. If you can do that without checking these notes, you have a useful foundation for studying OS algorithms.

These are teaching notes, not a substitute for the lecturer's exact terminology, diagrams, or any grading rubric. When the lecture uses a special definition, compare it with the general explanation here.